

# CS 354 - Lab 6: Bounded Buffer in Xinu

## Introducción / Introduction

En este laboratorio, implementamos un sistema de **bounded buffer** (búfer limitado) utilizando semáforos para coordinar múltiples productores y consumidores dentro del sistema operativo Xinu. El objetivo fue entender la sincronización de procesos y el control de concurrencia mediante mecanismos clásicos de semáforos.

In this lab, we implemented a **bounded buffer system** using semaphores to coordinate multiple producer and consumer processes within the Xinu operating system. The goal was to understand process synchronization and concurrency control through classical semaphore mechanisms.

## Temas Principales / Main Topics

- Sincronización de procesos / Process Synchronization
- Semáforos (wait, signal) / Semaphores (wait, signal)
- Concurrencia entre múltiples procesos / Concurrency among multiple processes
- Buffers circulares / Circular Buffers
- Manejo de recursos reutilizables / Reusable resource management

### ***bb.h***

Define la estructura global del búfer limitado y las funciones principales. Define the global bounded buffer structure and main function prototypes.

### ***bb\_create.c***

Inicializa el búfer, semáforos y variables necesarias. Initializes the buffer, semaphores, and all necessary variables.

### ***bb\_in.c***

Inserta bytes en el búfer, esperando si el búfer está lleno. Inserts bytes into the buffer, waiting if it is full.

### ***bb\_out.c***

Extrae bytes del búfer, esperando si está vacío o devolviendo EOF si terminó. Removes bytes from the buffer, waiting if empty or returning EOF if ended.

### ***bb\_end.c***

Marca el final de la transmisión de datos. Marks the end of data transmission.

### ***bb\_delete.c***

Libera la memoria y reinicia los semáforos para reutilizar el recurso. Frees memory and resets semaphores to reuse the resource.

### ***main.c***

Prueba todos los casos: productor/consumidor único y múltiples procesos. Tests all cases: single producer/consumer and multiple processes.

## Archivo / File: bb.h

```
typedef struct bounded_buffer {
    bool8 active;
    byte *buffer;
    int32 size;
    int32 head;
    int32 tail;
    bool8 endflag;
    sid32 mutex;
    sid32 empty;
    sid32 full;
} bounded_buffer_t;
```

## Archivo / File: bb\_create.c

```
int32 bb_create(int32 bufsize) {
    if (bb.active || bufsize <= 0) return SYSERR;
    bb.buffer = (byte *)getmem(bufsize);
    if (bb.buffer == (byte *)SYSERR) return SYSERR;
    bb.size = bufsize;
    bb.head = bb.tail = 0;
    bb.endflag = FALSE;
    bb.mutex = semcreate(1);
    bb.empty = semcreate(bufsize);
    bb.full = semcreate(0);
    bb.active = TRUE;
    return OK;
}
```

## Archivo / File: bb\_in.c

```
int32 bb_in(byte b) {
    if (!bb.active || bb.endflag) return SYSERR;
    wait(bb.empty);
    wait(bb.mutex);
    bb.buffer[bb.head] = b;
    bb.head = (bb.head + 1) % bb.size;
    signal(bb.mutex);
    signal(bb.full);
    return OK;
}
```

## Archivo / File: bb\_out.c

```
int32 bb_out(void) {
    byte b;
    if (!bb.active) return SYSERR;
    wait(bb.full);
    wait(bb.mutex);
    if (bb.endflag && bb.head == bb.tail) {
        signal(bb.mutex);
        signal(bb.full);
    }
}
```

```

        return EOF;
    }
    b = bb.buffer[bb.tail];
    bb.tail = (bb.tail + 1) % bb.size;
    signal(bb.mutex);
    signal(bb.empty);
    return b;
}

```

## Archivo / File: bb\_end.c

```

int32 bb_end(void) {
    if (!bb.active) return SYSERR;
    wait(bb.mutex);
    bb.endflag = TRUE;
    signal(bb.mutex);
    return OK;
}

```

## Archivo / File: bb\_delete.c

```

int32 bb_delete(void) {
    if (!bb.active) return SYSERR;
    freemem((char *)bb.buffer, bb.size);
    semdelete(bb.mutex);
    semdelete(bb.empty);
    semdelete(bb.full);
    bb.active = FALSE;
    bb.buffer = NULL;
    bb.size = 0;
    bb.head = bb.tail = 0;
    bb.endflag = FALSE;
    return OK;
}

```

## Archivo / File: main.c

```

process main(void) {
    bb_create(5);
    resume(create(producer, 4096, 20, "producer", 0));
    resume(create(consumer, 4096, 20, "consumer", 0));
    sleep(2);
    bb_delete();
    return OK;
}

```

## Conclusión / Conclusion

Este laboratorio demuestra el uso de semáforos para la sincronización de procesos y manejo de recursos compartidos. Los principios de exclusión mutua, espera activa y reutilización del recurso fueron aplicados correctamente.

This lab demonstrates the use of semaphores for process synchronization and shared resource management. The principles of mutual exclusion, blocking wait, and reusable resources were correctly implemented.